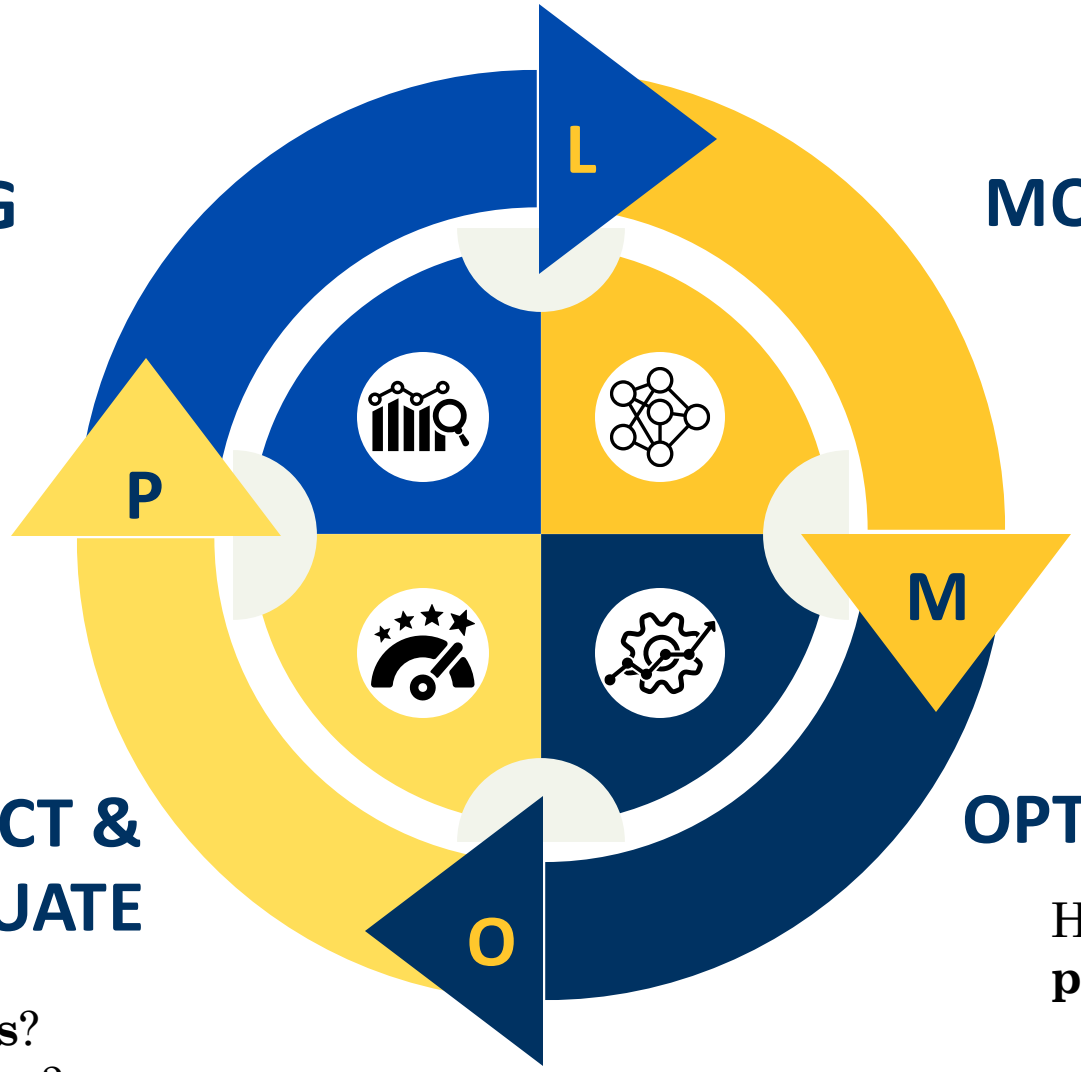
Discussion Mini Lecture 2

Machine Learning Design

The Supervised Learning Design Process

CS 189/289A, Fall 2025 @ UC Berkeley
Sara Pohland

Machine Learning Lifecycle



(1) LEARNING PROBLEM

What do I want to **predict**?
What **data** do I have?

MODEL DESIGN (2)

What **features** should I use?
What **model family**?

OPTIMIZATION (3)

How do I **learn** the **parameters** of my model?

(4) PREDICT & EVALUATE

How do I make **predictions**?
How do I **assess** performance?

Concepts Covered

1. Data Pre-processing
2. Model Design
   a) Feature Engineering
   b) Model Families
   c) Design Considerations
3. Fitting a Model
4. Making a Prediction

Understanding & Preparing our Data

1. Data Pre-processing
2. Model Design
   a) Feature Engineering
   b) Model Families
   c) Design Considerations
3. Fitting a Model
4. Making a Prediction

Desired Form of Data

What do we want our data to look like for training?

Dataset: $\mathcal{D} = \{(\boldsymbol{x}_n, y_n)\}_{n=1}^{N}$

N = number of samples

Input: $\boldsymbol{x}_n \in \mathbb{R}^D$

D = number of features

Output: $y_n \in \mathbb{R}$

May not be available

Design Matrix:

$$\mathbb{X} = \begin{bmatrix} - \boldsymbol{x}_1^T - \\ \vdots \\ - \boldsymbol{x}_N^T - \end{bmatrix} = \begin{bmatrix} x_{11} & \cdots & x_{1D} \\ \vdots & \ddots & \vdots \\ x_{N1} & \cdots & x_{ND} \end{bmatrix} \in \mathbb{R}^{N \times D}$$

Target Vector:

$$Y = \begin{bmatrix} y_1 \\ \vdots \\ y_N \end{bmatrix} \in \mathbb{R}^N$$

Starting Form of Data

What does our data initially look like?

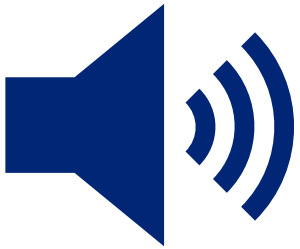
images

HI, It's your boss. Im stuck in Nigeria with none money. Please wire to TRWIGB2LXXX SOON.

text

# Bed	# Bath	…	Location
4	3	…	Berkeley

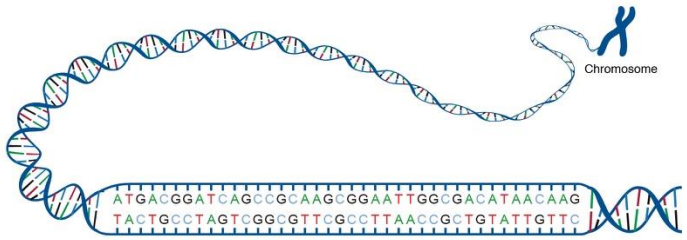
tabular



audio

time series

video



genomic

1) Investigate the Data

Practice investigating data: Section 2 of Discussion 2 notebook

Answer the following about your data:

- How many data samples do I have?
- What type of data am I working with?
- What does my data look like?
- Is my data labeled?

If you have categorical labels:

- How many unique labels are there?
- How many samples do you have from each class?

If you have quantitative labels:

- What is the distribution of the target labels?

2) Clean up the Data

Practice cleaning up tabular data: Sec. 2 of Discussion 2 notebook

For categorical features:

- Remove data with missing values or use mode imputation
- Use one-hot-encoding (see lecture 3)
  - This avoids artificial ordering that comes from simply converting a categorical features to a numeric value!

For numerical features:

- Remove data with missing values or use median imputation
- Normalize each feature (see lecture 3)
  - This avoids placing a disproportionate weight on features with larger magnitudes and improves numerical stability!
- Consider removing highly correlated variables

2) Clean up the Data

For text:

- Remove punctuation, special characters, stop words, etc.

For images:

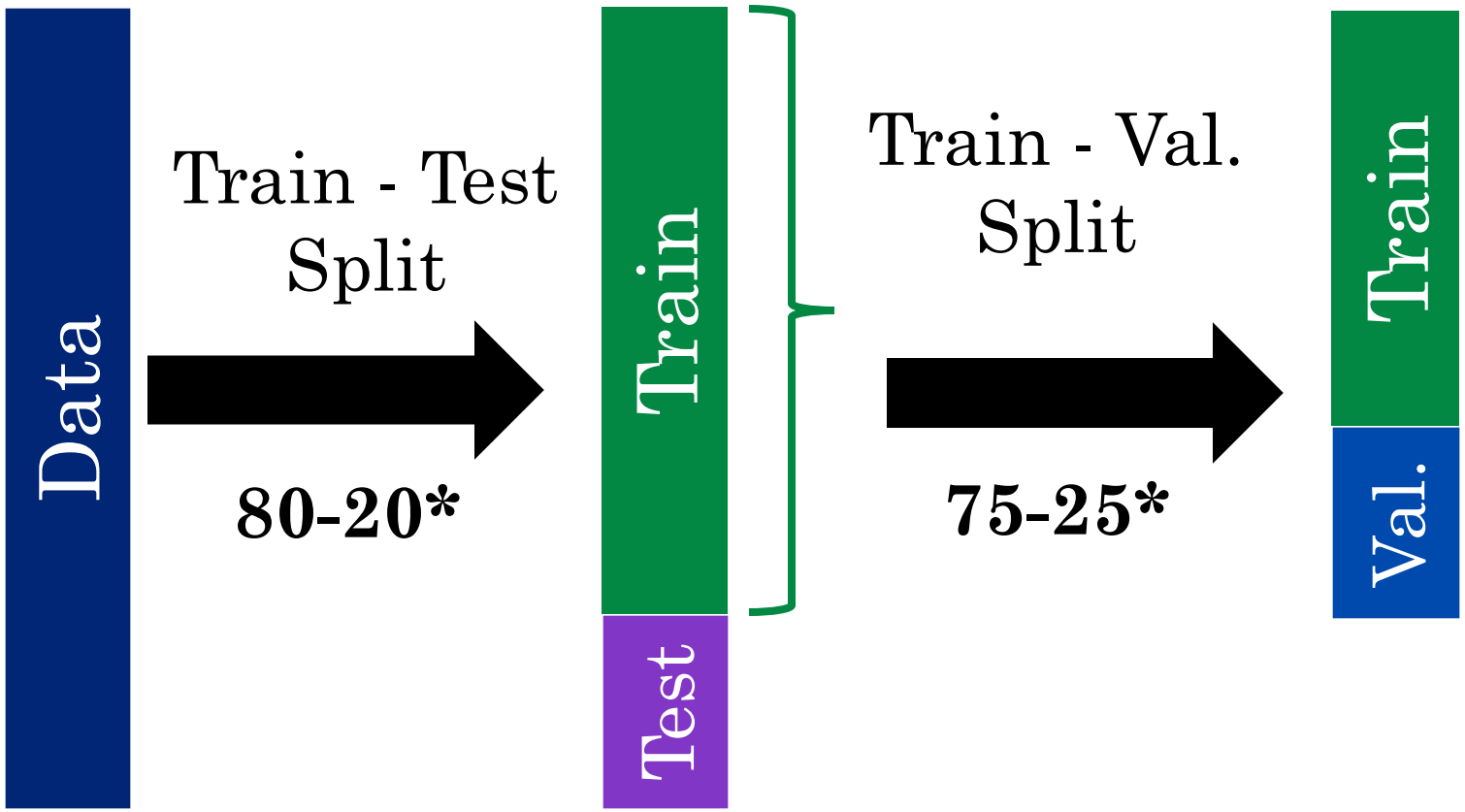
- Reshape images to a fixed size
- Normalize pixel values

For all data types:

- Remove “bad” data (low-quality images, anomalies, etc.)
- Consider removing features that we do not expect/want to help us predict our label/target
  - Features with poor predictive power can introduce unnecessary variance!

3) Divide Data into Three Sets

Practice splitting data into sets: Section 3 of Discussion 2 notebook



Train: ~60% of data

Used to fit the model during training

Val.: ~20% of data

Used to select model params. during training

Test: ~20% of data

Used to estimate performance post-training

*These are commonly used splits, but you can adjust these percentages based on your data.

Choosing/Learning Good Features

1. Data Pre-processing
2. Model Design
   a) Feature Engineering
   b) Model Families
   c) Design Considerations
3. Fitting a Model
4. Making a Prediction

Generating Features

For text data:

- Use a Bag-of-Words model
  - i.e., Count the presence of words in a vocabulary (see lecture 3)
  - Usually stop words (e.g., the, is, of…) that contain minimal info are dropped
- Count the use of punctuation marks
- Consider text length as a feature
- Look for specific patterns in text
  - e.g., Presence of URLs, typos, all caps words, etc.

Generating Features

For image data:

- Generate hand-crafted features
  - e.g., Use edge detectors to count presence of edges, texture descriptors to identify patterns, or CV tools to measure image properties (e.g., brightness)
  - Not common today but could be used to design more interpretable models.
- Find deep learning representations
  - e.g., Use (some component of) a pretrained model (ResNet, CLIP, etc.) to generate a feature vector that can be used for their ML task.
  - e.g., Use an object detection model to detect the presence of objects that might be relevant to your larger ML task.

Generating Features

Practice engineering features: Section 2 of Discussion 2 notebook

For tabular data:

- Derive features from existing ones
  - e.g., We want to predict worker burnout for staff members who started working in different weeks. Among other things, we're given the total number of hours each worked and total number of weeks. We also might want to derive the average number of hours worked per week.

For any data:

- Use domain knowledge to identify what information might be useful in predicting the thing we want.

Selecting a Model Family

1. Data Pre-processing
2. Model Design
   a) Feature Engineering
   b) Model Families
   c) Design Considerations
3. Fitting a Model
4. Making a Prediction

Determining the ML Paradigm

Do I have observations associated with my data?

- Yes, rewards → Reinforcement Learning
- Yes, labels → Supervised Learning
- No → Unsupervised Learning

What type of labels?

- Categorical → Classification
- Quantitative → Regression

What is my goal?

- Group data → Clustering
- Reduce features → Dimensionality Reduction

Examples of ML Problems

1. A botanist gives images of plants and tells me what they are. When I see a plant in the wild, I want to label it myself.

   → Categorical labels → Classification

2. Before testing medical interventions, I want to group together patients that have similar medical backgrounds.

   → Grouping data → Clustering

3. I want my robot to fold my laundry but don't know how to encode this. I reward it for every article of clothing it successfully folds.

   → Rewards → Reinforcement Learning

Examples of ML Problems

Practice identifying the ML paradigm: Disc. 2 worksheet

4. You give me a list of ratings for all of the movies you've watched. I want to predict how you want to rate a movie you haven't seen.

   → Quantitative labels → Regression

5. I have a table of data with 10,000 columns. I want to remove redundant features before training my model.

   → Reducing features → Dimensionality Reduction

Regression Model Families

Simple → **Complex**

Linear Regression
(Lecture 5)

Neural Networks
(Lecture 12)

Classification Model Families

Simple → **Complex**

Logistic Regression
(Lecture 7)

Neural Networks
(Lecture 12)

Interplay of Features and Model Families

1. Data Pre-processing
2. Model Design
   a) Feature Engineering
   b) Model Families
   c) Design Considerations
3. Fitting a Model
4. Making a Prediction

Selecting Features & Model Families

Key Considerations:

- Machine learning is full of trade-offs.
  - More features provide more information to our model, but they introduce additional variance, so we're more likely to capture spurious data patterns.
  - More complex models can capture more complex input-output relationships, but they're more likely to overfit to our training set.
  - Larger datasets can help us learn more generalizable models (assuming our dataset is representative of our population), but they require more memory and computational resources to train with them.
  - Engineering new features and increasing model complexity can improve accuracy, but this may also reduce interpretability.

Selecting Features & Model Families

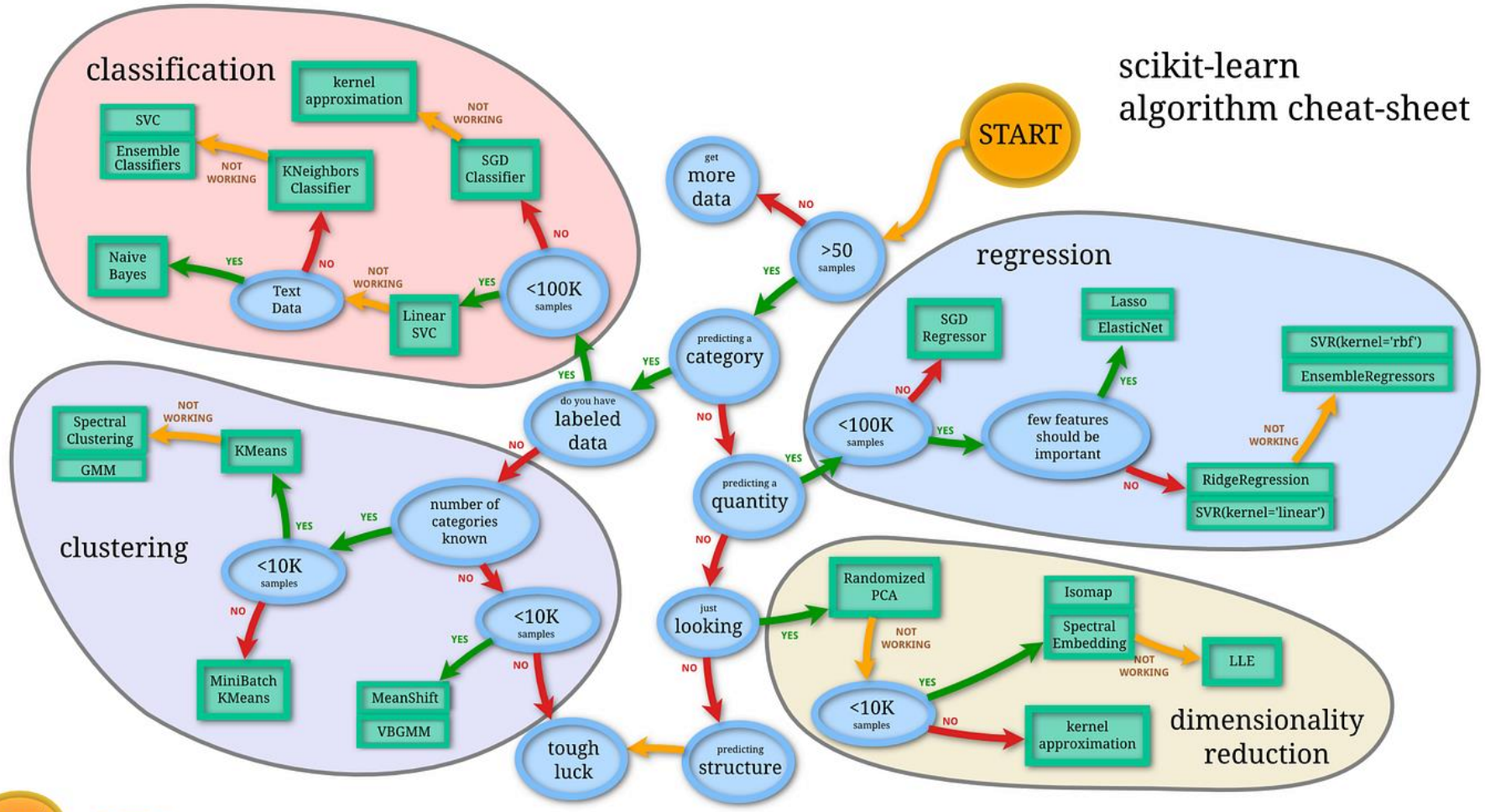
Key Considerations:

- Feature engineering and model selection are complementary.
  - If we have a simple model (e.g., a linear classifier), we probably want to choose/learn some new, useful features.
  - If we have a complex model (e.g., a neural network), we may want to work with our original, unmodified features.
- Domain knowledge is critical for training/deploying ML models.
- When in doubt, run validation to evaluate your options.

Learning Model Parameters

1. Data Pre-processing
2. Model Design
   a) Feature Engineering
   b) Model Families
   c) Design Considerations
3. Fitting a Model
4. Making a Prediction

Scikit-learn is Your Friend!



https://medium.com/@sumit.kaul.87/a-comprehensive-guide-to-scikit-learn-machine-learning-in-python-with-code-examples-8e4670877d03

Using Scikit-learn to Fit a Model

- Scikit-learn provides dozens of built-in machine learning algorithms and models, called estimators.
- Each estimator can be fitted to some data using its fit method.

```
>>> from sklearn.ensemble import RandomForestClassifier
>>> clf = RandomForestClassifier(random_state=0)
>>> X = [[ 1,  2,  3],  # 2 samples, 3 features
...      [11, 12, 13]]
>>> y = [0, 1]  # classes of each sample
>>> clf.fit(X, y)
```

Live, love, learn the fit method □

Using Scikit-learn to Fit a Model

- The fit method generally accepts 2 inputs:
  - The $N \times D$ design matrix, X, where samples are represented as rows and features are represented as columns (as in our notation).
  - The N target values, y, which are real numbers for regression tasks and a discrete set of values for classification. It is usually a 1D array where the nth entry corresponds to the target of the nth sample (row) of X. (For unsupervised learning tasks, y does not need to be specified.)
- Both X and y are usually expected to be NumPy arrays or equivalent array-like data types.

Practice fitting a model: Section 4 of Discussion 2 notebook

Making Model Predictions

1. Data Pre-processing
2. Model Design
   a) Feature Engineering
   b) Model Families
   c) Design Considerations
3. Fitting a Model
4. Making a Prediction

Using Scikit-learn to Make Predictions

- After using Scikit-learn to fit an estimator, it can be used for predicting target values of new data.

```
>>> clf.predict(X)  # predict classes of the training data
array([0, 1])
>>> clf.predict([[4, 5, 6], [14, 15, 16]])  # predict classes of new data
array([0, 1])
```

With fit comes its forever +1, predict

Practice making predictions: Sec. 4 of Discussion 2 notebook

Discussion Mini Lecture 2

Machine Learning Design

Contributors: Sara Pohland

Additional Resources

1. Fitting a model and making predictions
   - Getting Started with Scikit-learn
2. We'll go deeper into other things later in the semester!